

Module 01: Object-Oriented Programming

Language: Java**Type:** Pre-lab**Mode:** individual practical work

Learning outcomes

By completing this module, you should be able to:

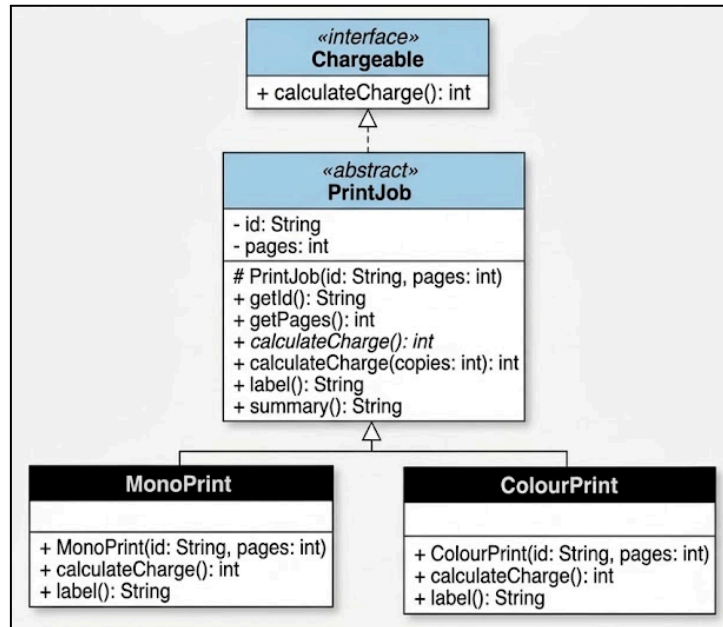
1. Distinguish a class, an object, and a reference variable.
2. Encapsulate state and initialise objects through constructors.
3. Implement inheritance, abstract classes, and an interface.
4. Distinguish method overloading from method overriding.
5. Explain and demonstrate runtime polymorphism through parent and interface references.
6. Test behaviour and justify a design using concrete examples from your code.

Concept Toolkit

Concept	Meaning and a useful distinction
Class and object	A class defines state and behaviour. Each object is an instance with its own identity and instance fields. A reference variable refers to an object; it is not the object itself.
Encapsulation	Keep fields private and expose appropriate operations. A getter can allow reading without allowing arbitrary external modification.
Inheritance	extends expresses an “is-a” relationship and allows a subclass to reuse accessible parent behaviour. Constructors are not inherited.
Overloading	Methods have the same name but different parameter lists. A different return type alone is insufficient. Overload selection uses compile-time types.
Overriding	A subclass supplies a new implementation of an inherited instance method with a matching signature and compatible return type. Use <code>@Override</code> .
Polymorphism	A parent or interface reference can refer to a subtype object. For an overridden instance method, the object’s runtime class determines the implementation executed.
Abstract class	Cannot be instantiated directly. May contain fields, constructors, concrete methods, and abstract methods. A concrete subclass must fulfil remaining abstract obligations.
Interface	Specifies a contract that implementing classes must fulfil. In this lab, its method is implicitly public abstract. Java interfaces can also have other kinds of methods; they are not used here.

Prelab Case: Campus printing service

A campus print shop accepts monochrome and colour print jobs. Each job has an identifier and a positive number of pages. Different jobs calculate charges differently, but a client should request a charge through the same contract. All amounts below are integer rupiah (IDR).



Exact Behaviour

- `PrintJob` implements `Chargeable`; both concrete classes extend `PrintJob`. Initialise inherited state using `super(...)`. No setters are required.
- `MonoPrint`: IDR 500 per page.
- `ColourPrint`: first 10 pages at IDR 1,500 each; every page beyond 10 at IDR 1,000; add IDR 2,000 setup per complete copy.
- `calculateCharge(int copies)` returns `copies * calculateCharge()`. Implement this overload once in `PrintJob`; do not duplicate it in the subclasses.
- Parent `label()` returns "Print"; subclass labels are "Mono" and "Colour".
- `summary()` returns `id + " | " + label() + " | " + calculateCharge()`. Do not override `summary()`.
- Throw `IllegalArgumentException` if pages or copies are zero or negative. Assume valid non-empty IDs, pages at most 100, and copies at most 10. No additional upper-bound or ID validation is required.
- Input File (`jobs.txt`):
 - Input is read from a text file named `jobs.txt`.
 - Each line represents one print job with 3 space-separated values: `<TYPE> <ID> <PAGES>`.
 - There is no header line or total count line at the start of the file.
 - `<TYPE>` can be MONO or COLOUR.
 - `<ID>` is the job identifier (e.g., P01).
 - `<PAGES>` is a positive integer page count.

Labwork 01**Prelab Tasks****A1. Implementation and demonstration**

1. Implement the Chargeable interface and all three classes (PrintJob, MonoPrint, and ColourPrint). Add @Override to each subclass override and to the abstract declaration implementing the interface method.
2. In Main, read jobs.txt using Scanner (scanner.next(), scanner.nextInt()).
3. Map each line based on its type into a concrete MonoPrint or ColourPrint object.
4. Store the created objects in a collection of PrintJob references (e.g., List<PrintJob> or PrintJob[]).
5. Print each job's summary() using a single loop through the PrintJob references. Do not select behaviour with instanceof, casts, or type-name conditionals (demonstrating runtime polymorphism).

A2. Required checks

Input file jobs.txt:

```
MONO P01 8
COLOUR P02 12
COLOUR P10 10
COLOUR P11 11
MONO P12 1
```

The output of Main must produce the following summaries:

Job / Input	Rate Calculation	Expected summary ()
MONO P01 8	8 pages × IDR 500	P01 Mono 4000
COLOUR P02 12	$(10 \times 1,500) + (2 \times 1,000) + 2,000$	P02 Colour 19000
COLOUR P10 10	$(10 \times 1,500) + 2,000$	P10 Colour 17000
COLOUR P11 11	$(10 \times 1,500) + (1 \times 1,000) + 2,000$	P11 Colour 18000
MONO P12 1	1 page × IDR 500	P12 Mono 500

The program execution must display exactly:

```
P01 | Mono | 4000
P02 | Colour | 19000
P10 | Colour | 17000
P11 | Colour | 18000
P12 | Mono | 500
```

Labwork 01

Submission

Please follow these instructions carefully to submit your work:

1. Create the `lw01` folder under your `dsa-[nrp]` repository, the repository you created previously in Module 00: Java Setup (check: [0. Java Environment Setup](#)). Inside the `lw01` folder, create a subfolder named `prelab`.
2. Complete all of your work for this prelab inside the `prelab` folder.
3. Commit and push your work to GitHub with the commit message:
"`Labwork 01 - Prelab`"
4. **Deadline: Monday, September 21, 2026, at 6:00 AM (WIB).**

To give a clearer picture, The repository for this semester's practicum will follow the structure below:

```
dsa-[nrp]/
├──
└── src/
    ├── lw01/
    │   ├── prelab/
    │   │   └── [your prelab files]
    │   └── unguided/
    │       └── [your unguided files]
    ├── lw02/
    │   ├── prelab/
    │   │   └── [your prelab files]
    │   └── unguided/
    │       └── [your unguided files]
    ├── lw03/
    │   ├── prelab/
    │   │   └── [your prelab files]
    │   └── unguided/
    │       └── [your unguided files]
    └── ...
```